SOMAIYA
VIDYAVIHAR
K J Somaiya Institute of Technology
An Autonomous Institute Permanently Affiliated to the University of Mumbai

DEPARTMENT OF INFORMATION TECHNOLOGY

Course Name and Code: Data Structures Lab **(**ITL302)
Semester: III (SYIT)
Academic Year: 2026-27 (Odd Semester)
Name: Rashi Dawda
Roll No: 23
Batch: AS2

Experiment No. 02

Aim: Implementation of Queue using an array for real-world application.

Objectives:

1. To introduce the concepts of data structures and analysis procedure.
2. To conceptualize linear data structures and its implementation for various real-world applications.

Theory:

1. Introduction to Queue

A **Queue** is a linear data structure that works on the **FIFO (First In, First Out)** principle — the first element added is the first one to be removed. A simple way to picture this is a queue of people waiting in line: the person who joins first gets served first, and new people join from the back. Insertion happens from one end, called the **rear**, and deletion happens from the other end, called the **front**. It can be implemented using an **array** (fixed size, with `front` and `rear` variables tracking the ends) or a **linked list** (which can grow and shrink dynamically). In this experiment, the array-based method is used, where `MAX` defines the maximum number of elements the queue can hold, and `front` and `rear` start at `-1` to indicate that the queue is initially empty.

2. Various Operations

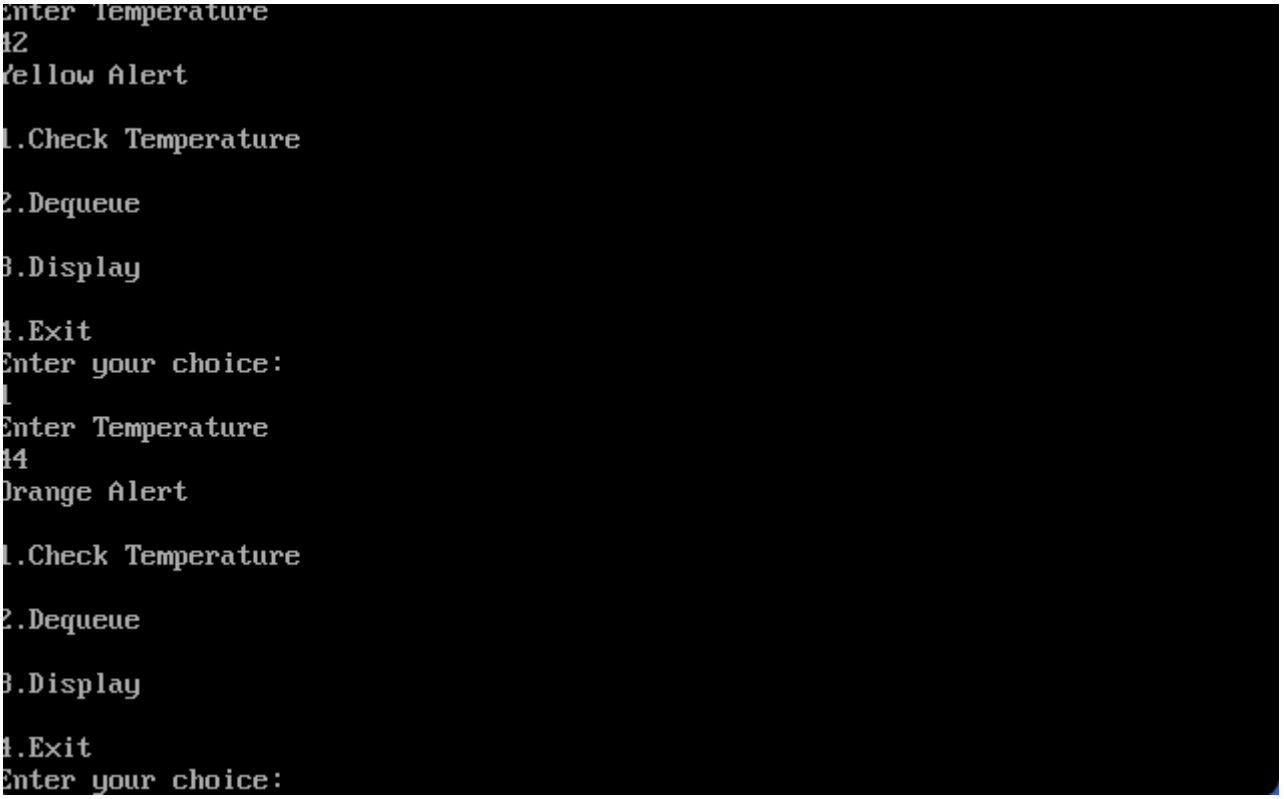
- **ENQUEUE** – Adds a new element to the rear of the queue. If the queue is already full (`rear == MAX-1`), it results in a **queue overflow**.
- **DEQUEUE** – Removes the element from the front of the queue. If the queue is empty (`front == -1` or `front > rear`), it results in a **queue underflow**.
- **PEEK** – Lets you view the value of the front element without actually removing it from the queue.
- **DISPLAY** – Prints all the elements currently present in the queue, from front to rear.

Conclusion:

Doing this experiment helped me actually understand how a queue works instead of just reading about FIFO in theory. Writing the ENQUEUE and DEQUEUE functions made it clear why we need to check for overflow and underflow before adding or removing elements. The heatwave alert example was a nice touch too, since it showed how something as simple as a queue can be used to process temperature readings in order and flag alerts in a real system.

Outcome:

Apply the concepts of queues for real-world applications.



```
1
Enter Temperature
44
Orange Alert

1.Check Temperature

2.Dequeue

3.Display

4.Exit
Enter your choice: 1
Enter Temperature
46
Red Alert

1.Check Temperature

2.Dequeue

3.Display

4.Exit
Enter your choice:
```

```
Enter Temperature
46
Red Alert

1.Check Temperature

2.Dequeue

3.Display

4.Exit
Enter your choice: 3
Active Alerts
Yellow Alert
Orange Alert
Red Alert

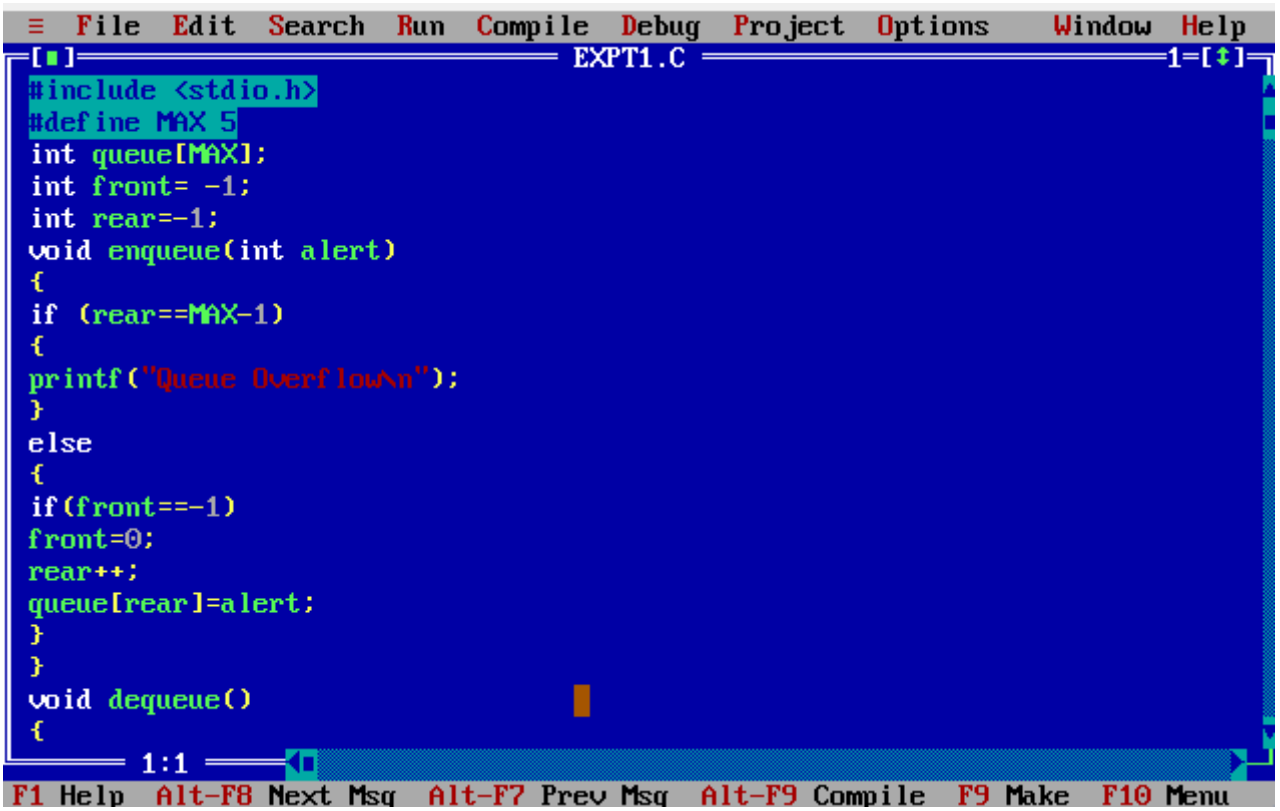
1.Check Temperature

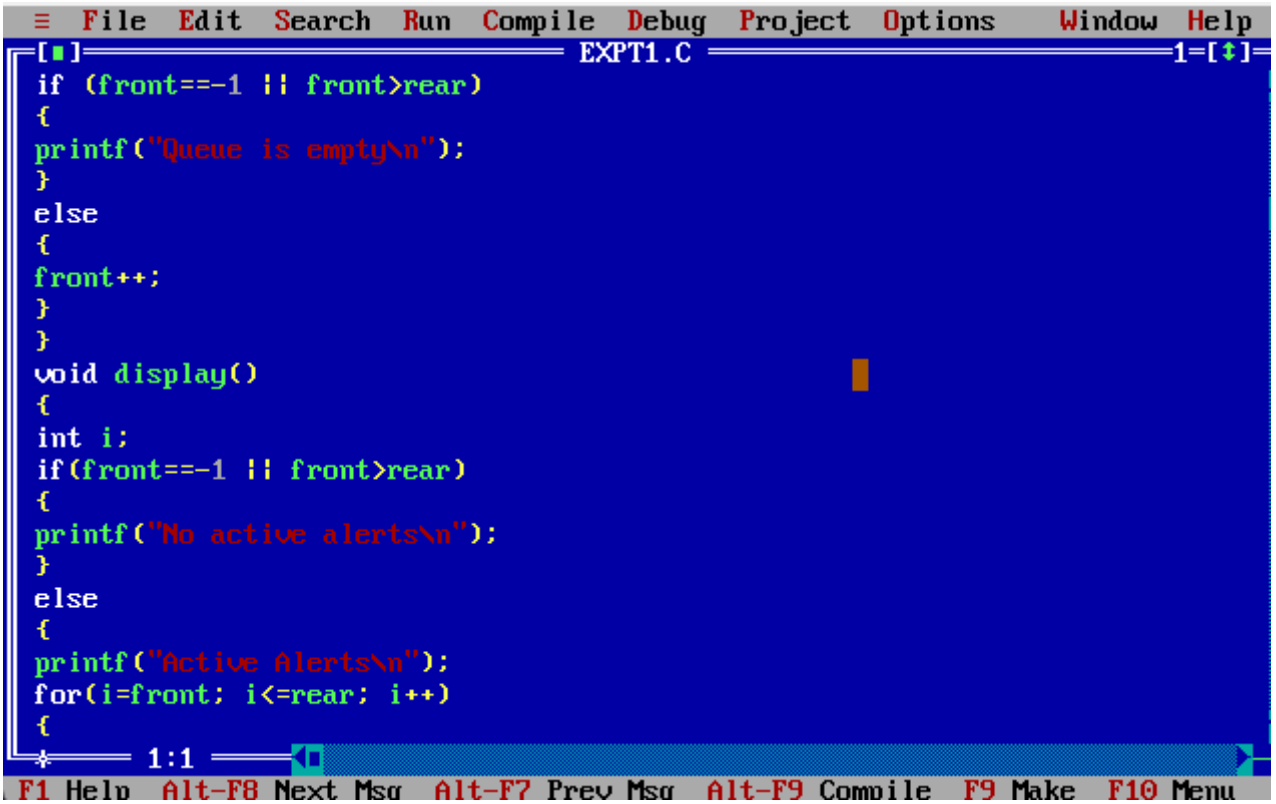
2.Dequeue

3.Display

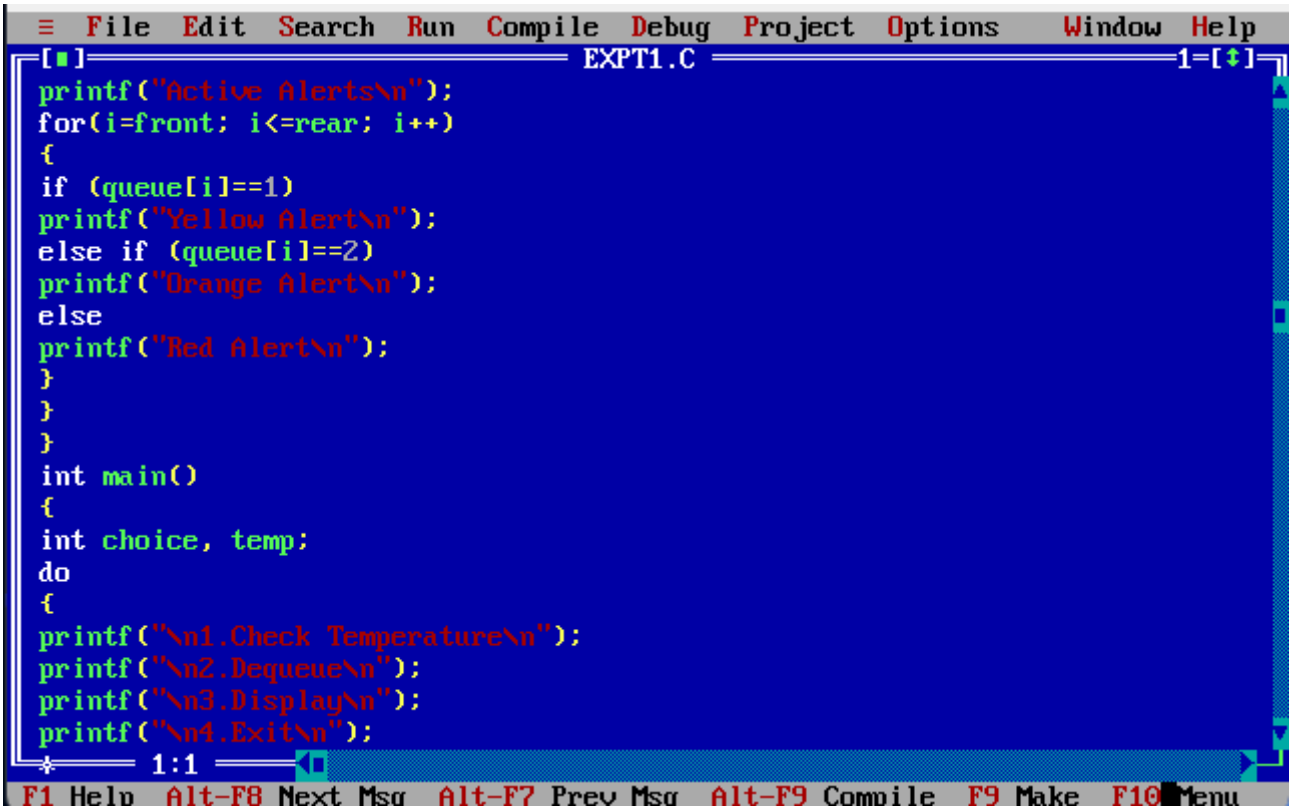
4.Exit
Enter your choice:
```

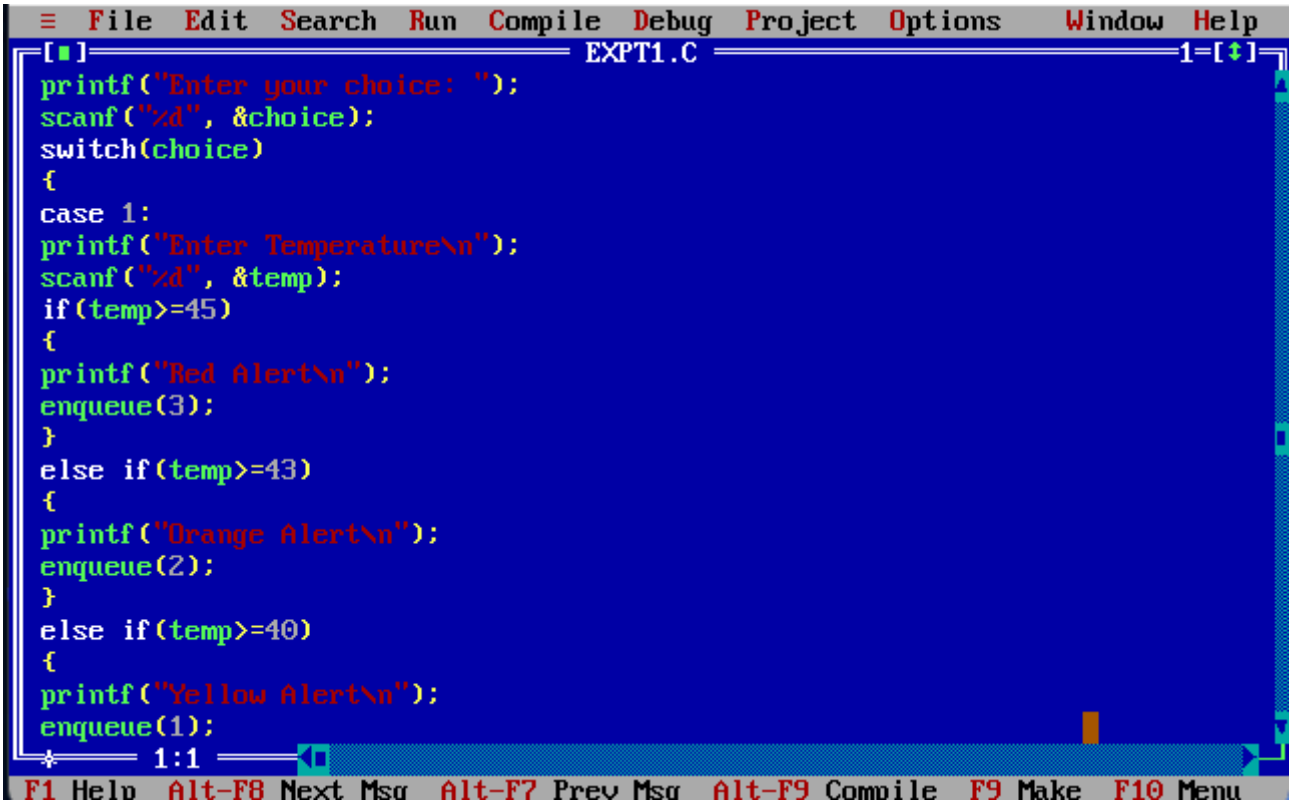
CODE:


File Edit Search Run Compile Debug Project Options Window Help
EXPT1.C
#include <stdio.h>
#define MAX 5
int queue[MAX];
int front= -1;
int rear=-1;
void enqueue(int alert)
{
if (rear==MAX-1)
{
printf("Queue Overflow\n");
}
else
{
if(front==-1)
front=0;
rear++;
queue[rear]=alert;
}
}
void dequeue()
{
1:1
F1 Help Alt-F8 Next Msg Alt-F7 Prev Msg Alt-F9 Compile F9 Make F10 Menu




```
if (front==-1 || front>rear)
{
printf("Queue is empty\n");
}
else
{
front++;
}
}
void display()
{
int i;
if(front==-1 || front>rear)
{
printf("No active alerts\n");
}
else
{
printf("Active Alerts\n");
for(i=front; i<=rear; i++)
{
```


File Edit Search Run Compile Debug Project Options Window Help
EXPT1.C
printf("Active Alerts\n");
for(i=front; i<=rear; i++)
{
if (queue[i]==1)
printf("Yellow Alert\n");
else if (queue[i]==2)
printf("Orange Alert\n");
else
printf("Red Alert\n");
}
}
}
int main()
{
int choice, temp;
do
{
printf("\n1.Check Temperature\n");
printf("\n2.Dequeue\n");
printf("\n3.Display\n");
printf("\n4.Exit\n");
1:1
F1 Help Alt-F8 Next Msg Alt-F7 Prev Msg Alt-F9 Compile F9 Make F10 Menu


File Edit Search Run Compile Debug Project Options Window Help
EXPT1.C
printf("Enter your choice: ");
scanf("%d", &choice);
switch(choice)
{
case 1:
printf("Enter Temperature\n");
scanf("%d", &temp);
if(temp>=45)
{
printf("Red Alert\n");
enqueue(3);
}
else if(temp>=43)
{
printf("Orange Alert\n");
enqueue(2);
}
else if(temp>=40)
{
printf("Yellow Alert\n");
enqueue(1);
1:1
F1 Help Alt-F8 Next Msg Alt-F7 Prev Msg Alt-F9 Compile F9 Make F10 Menu


File Edit Search Run Compile Debug Project Options Window Help
EXPT1.C
else
{
printf("Temperature below 40\n");
while(front!=-1 && front<=rear)
{
dequeue();
}
front =-1;
rear=-1;
}
break;
case 2:
dequeue();
break;
case 3:
display();
break;
case 4:
1:1

File Edit Search Run Compile Debug Project Options Window Help

EXPT1.C

```
display();
break;

case 4:
printf("Exit\n");
break;

default:
printf("Invalid choice\n");
}
}while (choice!=4);
return 0;
}
```

1:1

File Edit Search Run Compile Debug Project Options Window Help

EXPT1.C

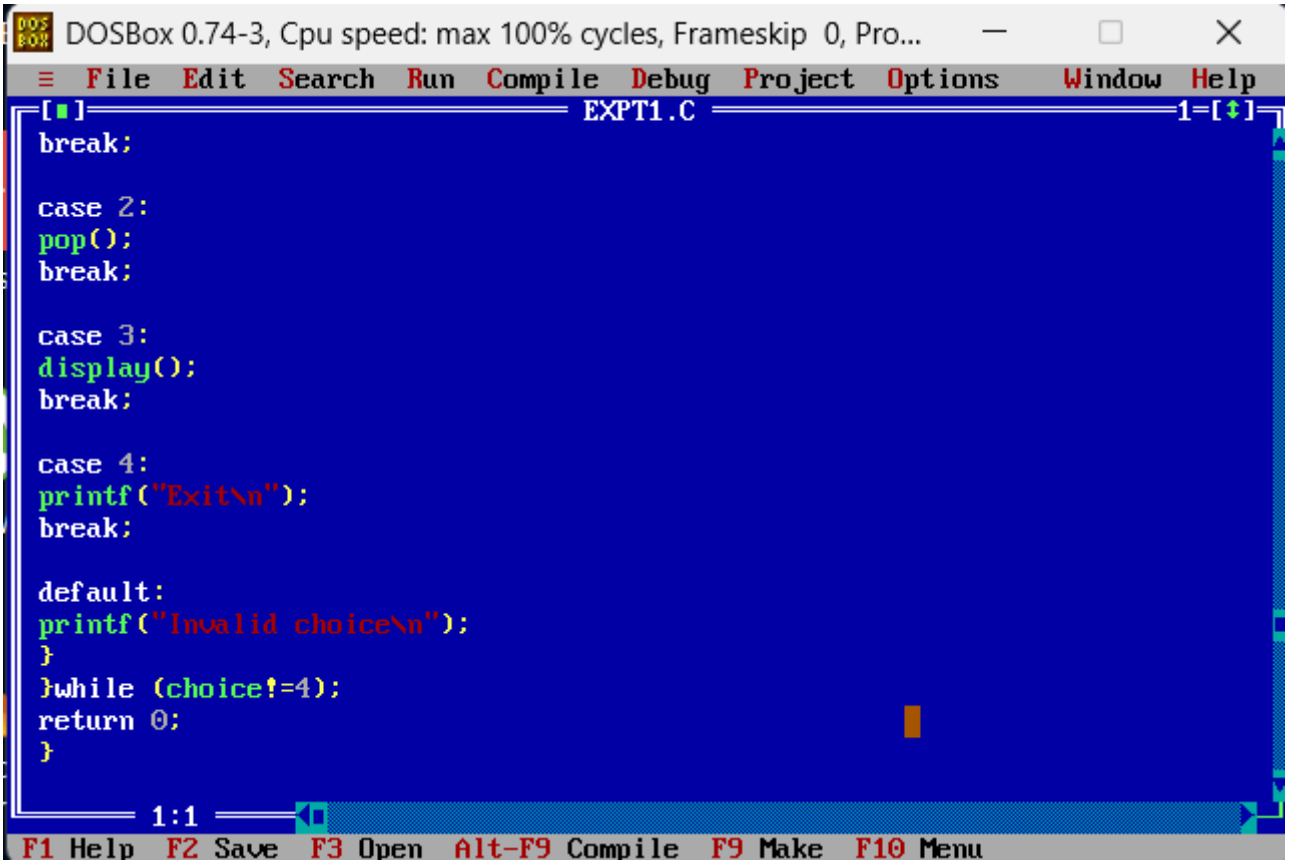
```
else
{
printf("Temperature below 40\n");
while(front!=-1 && front<=rear)
{
dequeue();
}
front =-1;
rear=-1;
}
break;

case 2:
dequeue();
break;

case 3:
display();
break;

case 4:
```

1:1


DOSBox 0.74-3, Cpu speed: max 100% cycles, Frameskip 0, Pro...
File Edit Search Run Compile Debug Project Options Window Help
EXPT1.C
break;
case 2:
pop();
break;
case 3:
display();
break;
case 4:
printf("Exit\n");
break;
default:
printf("Invalid choice\n");
}
}while (choice!=4);
return 0;
}
1:1
F1 Help F2 Save F3 Open Alt-F9 Compile F9 Make F10 Menu